HolmesGPT

Investigating incidents
Acting with clear limits

Observability for the LLM

Cédric DELGEHIER

01

Why HolmesGPT?

An alert reports a symptom. Someone still needs to choose the checks, read the results and explain what is happening before deciding what to do.

How do we turn an alert into a clear investigation and then a controlled repair?

HolmesGPT, an open source investigation agent

Originally created by **Robusta.dev**, HolmesGPT joined the **Cloud Native Computing Foundation** as a Sandbox project on **8 October 2025**.

It helps diagnose incidents through existing tools: metrics, logs, traces, service state and change history. Its scope can include Kubernetes, VMs, cloud services and databases.

The investigation engine and the language model are separate components. You can configure the LLM provider. The model chooses checks and interprets the results of the tools available to Holmes.

The goal is to reduce investigation work and provide an answer supported by evidence. Diagnosis time and quality need to be measured on real incidents.

What starts an investigation?

An investigation can start with an alert, a human question, a deployment or a scheduled check.

Trigger	Example
After an alert	A relay receives the alert and sends the incident context to Holmes.
On demand	An operator asks a question through the CLI or a connected interface.
After a deployment	Compare errors, latency and logs before and after the change.
On a schedule	Check a specific service regularly for signs of failure.

In the demo VM, Alertmanager starts investigations through our HTTP relay.

What Holmes adds to an alert

The starting point

An alert saying "HTTP service unavailable" means a check failed. It does not explain whether the process stopped, a dependency failed or a file became unreadable.

Without an automated investigation, a person must select the sources, run checks, read the results and decide what to do next.

What we expect from Holmes

Holmes reads the available metrics, system state and logs, then compares this evidence with possible causes.

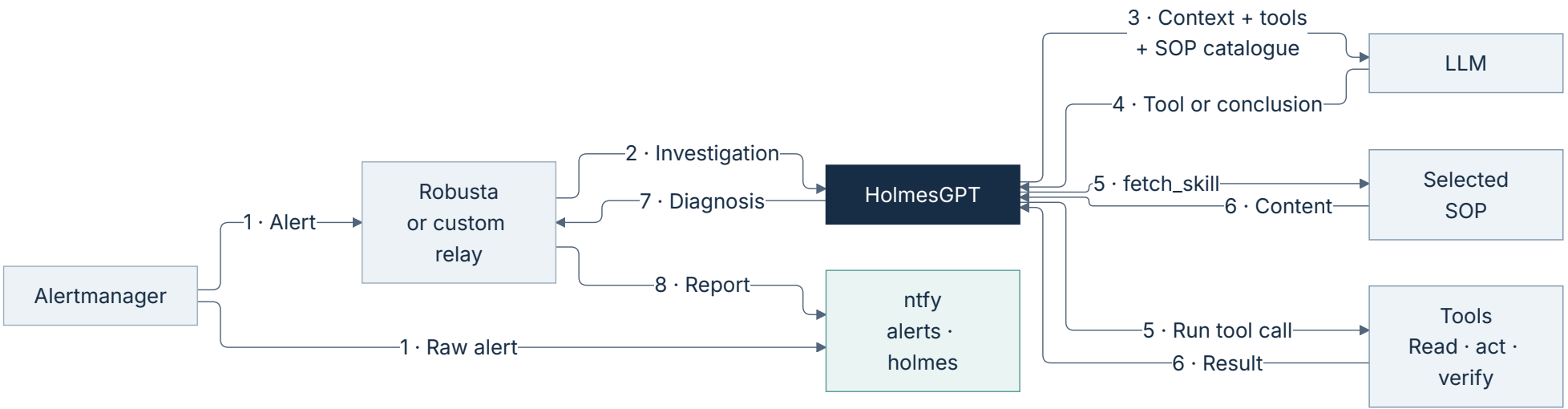
If a **Standard Operating Procedure (SOP)** exists, it guides the checks and the conditions for action. Otherwise, the model chooses checks based on the alert and the available tool results.

The report should reduce the work left for the operator. A repair requires an available tool, and the request must pass its checks.

Judge the investigation and the proposed next steps. Repeating the alert adds little value.

After an alert

Holmes sends the context and catalogues to the LLM. The model requests a tool, including `fetch_skill` for an SOP, then uses its result to decide the next step.



Loop 3–6 repeats: the loaded SOP and tool results become part of the context sent to the LLM. It can request another tool or conclude, within `MAX_STEPS`.

This POC uses a custom relay. ntfy topics: `alerts` for raw alerts; `holmes` for reports.

Model decisions and tool limits

A script could handle the known faults in this demo.
This POC shows how Holmes uses evidence and prepares the next steps for an incident.

The model leads the investigation

It suggests possible causes and requests checks with the available tools. The SOP describes the expected checks and the paths to follow.

It interprets results, requests an allowed action or explains why a human must take over. Its report should cite the observations that support its conclusion.

Tools and permissions set the limits

Permissions define the accessible resources and possible actions. **Tool controls can reject a model request.**

In this demo, our Python relay takes the place of Robusta for integration. It also checks the incident, required observations and target before acting, then verifies the result.

We assess investigation quality and the handover to humans. Repairing a known fault alone does not prove that an LLM works better than a script.

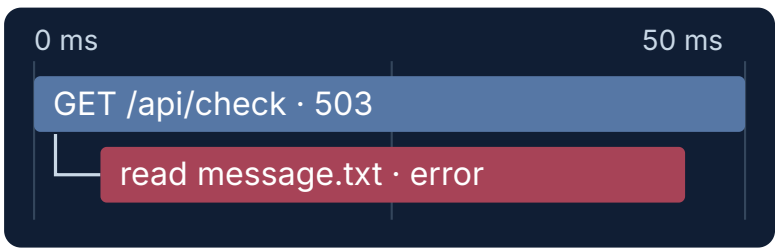
02

Understanding the investigation engine

Holmes connects a language model to tools. The model chooses the checks, and the tools provide real observations.

We have followed the alert. Now we look at the evidence that tools can collect.

What each source shows

Source	What we observe	Illustrative example
Metrics Prometheus Values tracked over time		The error rate rises from **0 to 40%**. We see when the service gets worse and how large the impact is.
Logs Loki Events with timestamps		The log reports **denied access to the file**. It provides the error message and its timestamp.
Traces Jaeger · OTel The operations in a request		The request fails while **reading the file**. We connect the error to its operation, duration and parent request.

Current state: systemd service state, Linux mounts and permissions, or pod status from the Kubernetes API.

03

Building the demo environment

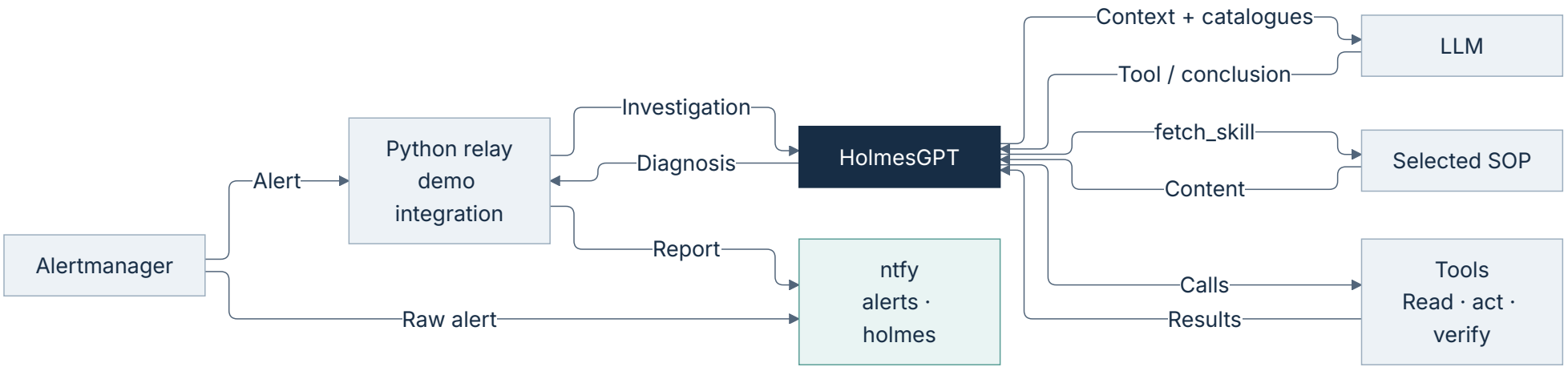
A Linux VM runs an HTTP service, observability collection and the relay to Holmes. An API provides access to the model server.

The demo follows the application, observations, alert, investigation and report.

The VM connects the application, evidence and Holmes

FastAPI serves the home page and `/api/check`, which reads the demo file.

Observability: Prometheus, Alertmanager, Blackbox Exporter, Node Exporter, OpenTelemetry Collector, Jaeger, Loki and Grafana.



Tools query Prometheus, Loki, Jaeger and Linux state. Grafana lets people explore these signals. Holmes returns the diagnosis to the relay, which publishes it in ntfy.

Logs and traces: two views of a request

A log describes an event

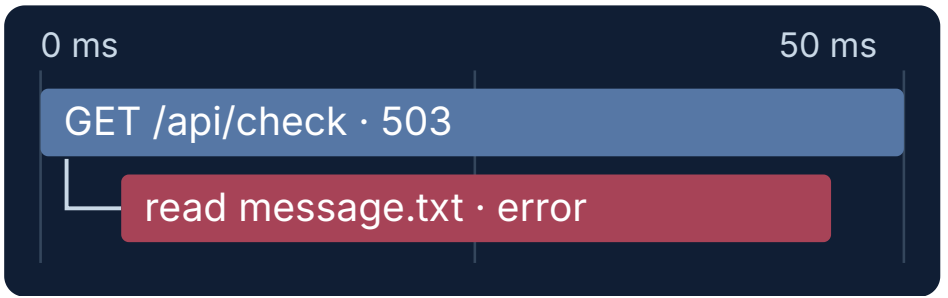
The FastAPI application writes an event with a timestamp, message, level and fields. Here, it explains why the file read failed.

```
{
  "message": "Demo file cannot be read",
  "error_type": "PermissionError",
  "trace_id": "…"
}
```

This message can be very precise. It does not automatically describe every operation in the request or the links between them.

A trace shows the request path

A trace contains **spans**. A span represents an operation with a start, an end, a status and an optional parent.



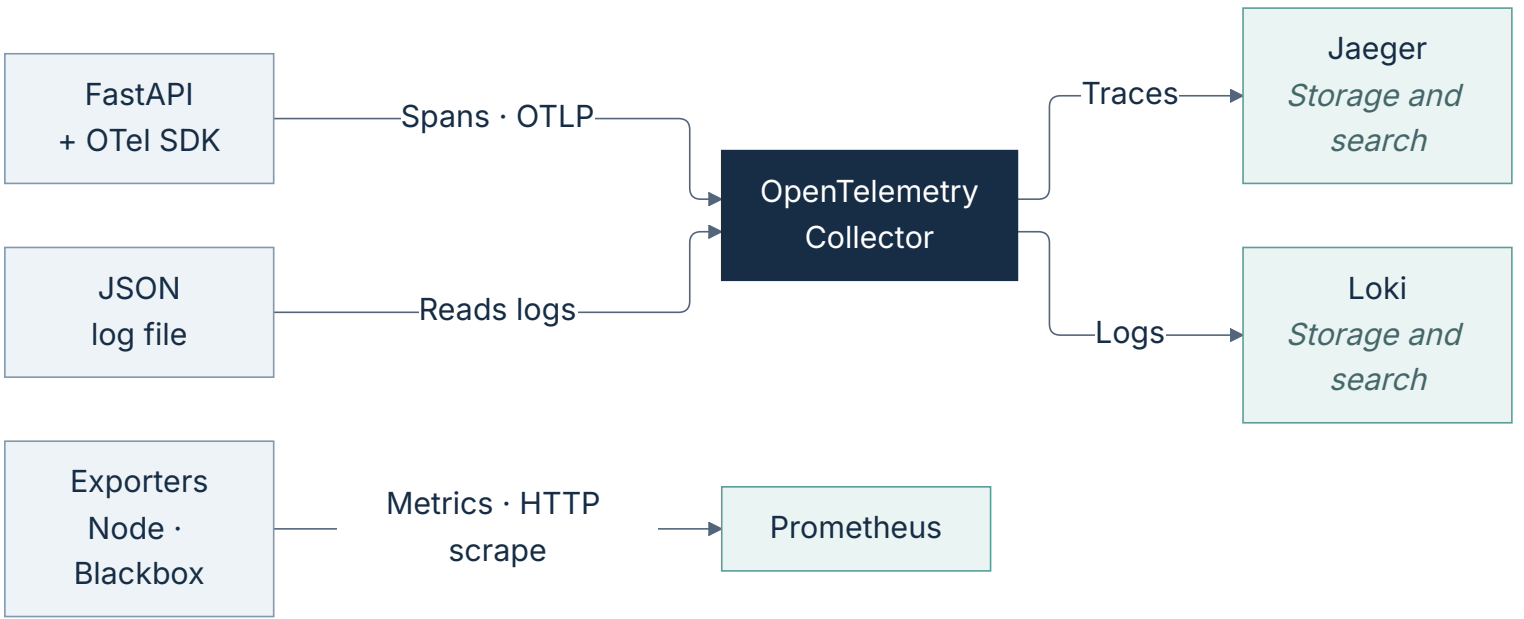
The HTTP bar is the **parent span**. The file read is its **child span**. The trace shows where the operation failed within the request.

A shared `trace_id` connects the detailed log to the request trace.

From signals to observability backends

```
with tracer.start_as_current_span("read demo message"):
    message = message_file.read()
```

FastAPI instrumentation creates the HTTP span. This code adds a **child span**. The SDK exports spans in the background.



The Collector reads JSON logs and the systemd journal. **Prometheus scrapes exporters:** Node Exporter measures the host, and Blackbox Exporter probes FastAPI.

04

Following an investigation

The HTTP probe fails while FastAPI is still running. We follow a file read error from the first symptom to the result of the intervention.

What evidence allows a repair, and which checks confirm that the service has recovered?

The probe fails while the process keeps running

In the demo, FastAPI reads `/srv/demo/message.txt` to answer the request. We remove read permissions without stopping the process.

```
# Inside the VM
sudo chmod 000 /srv/demo/message.txt
```

FastAPI stays active and serves its home page, but `/api/check` now returns HTTP 503. Its **log** reports `PermissionError`, and its **trace** shows the failed file read. Prometheus detects the failed probe, then Alertmanager sends the alert to the relay.

The alert reports an HTTP failure. The investigation still needs to check whether the process stopped or a file permission needs to change.

From evidence to Holmes' decisions

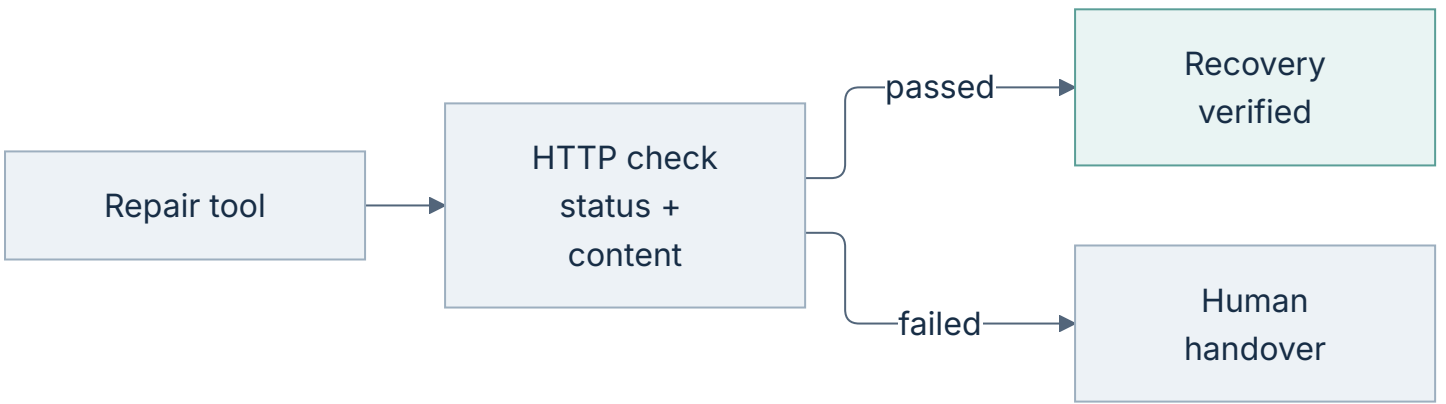
Each observation helps test a possible cause and choose the next check.

Evidence collected	What Holmes can conclude	Next check
Probe: HTTP `503`	The service responds, but request processing fails.	Find the failed operation in the trace.
Failed `read demo message` span	The file read fails. Its cause still needs checking.	Read the log with the same `trace_id`.
Log: `PermissionError`	The service cannot access the file.	Check the file, mount and service user.
File exists, mode `000`, mount and service active	The service cannot read the file.	Check the SOP and the allowed repair tool.

Conclusion: a permissions fault. The SOP and the tool define the repair conditions.

A successful action still needs a recovery check

A command may succeed while the incident continues. A process can start with an unavailable dependency or return the wrong HTTP content. After the action, we still need to check the service.



In the demo, the repair tool sends a new request and checks **HTTP 200 and the expected content**. If this check fails, Holmes should report "action completed, recovery unconfirmed" instead of declaring the incident resolved.

05

Controlling the agent

We have seen one possible repair. We still need to define when it is allowed, when the agent must stop and which resources its investigation can use.

The SOP guides decisions. Tools and permissions enforce the limits.

Inside an SOP: defining the investigation

An SOP should help Holmes recognise the right incident, choose useful checks and understand what their results can prove.

What to define	Example for an HTTP service
Trigger and scope	Alert name, target service and environment, owner and exceptions such as planned maintenance.
Expected behaviour	HTTP 200 and the expected content. A running process alone does not prove that the service is healthy.
Possible causes and evidence	Which checks separate a stopped service, a failed dependency and an unreadable file? What is still unknown?
Tools and collection	Name the tools, time window and useful fields. Limit data volume while keeping errors visible.

Example: an SOP for the HTTP service

```
---
name: demo-http-service
description: Diagnose DemoHttpUnavailable.
---
1. Read status, metrics and logs.
2. Confirm: failed probe AND stopped service.
3. Call start_demo_fastapi once if allowed.
4. Report the verified HTTP result.
```

Shortened extract from the HTTP service SOP

Keep the SOP in version control. Test it on a fault and on a case where the tool must refuse the action.

One SOP, one file

```
sops/
├── http-service/SKILL.md
├── file-permissions/SKILL.md
├── cpu-pressure/SKILL.md
├── memory-pressure/SKILL.md
├── disk-space/SKILL.md
├── missing-mount/SKILL.md
└── monitoring/SKILL.md
```

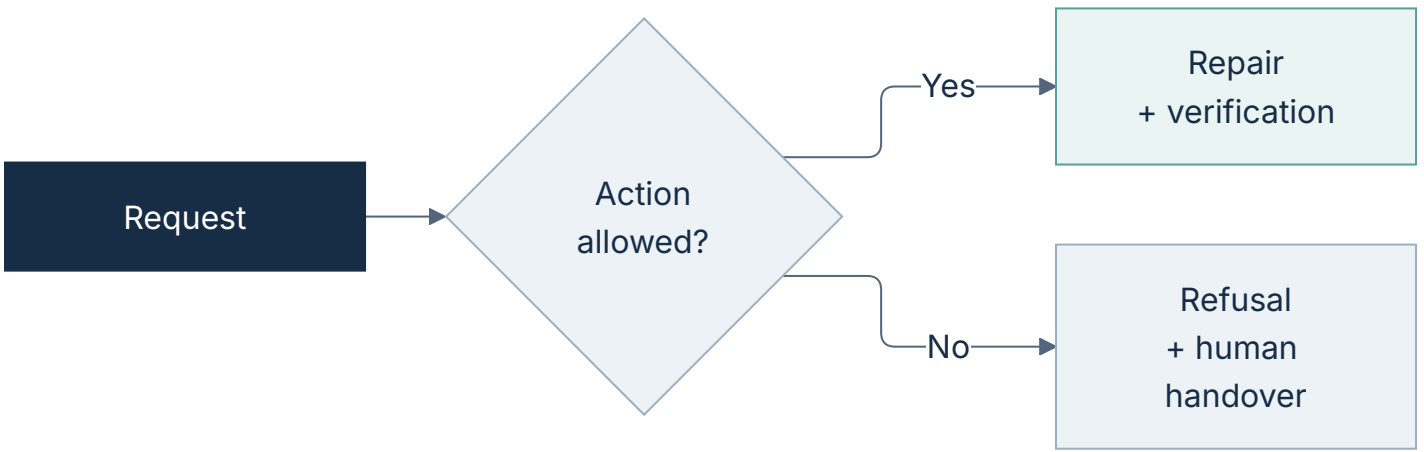
The name and description guide selection. The body defines evidence, allowed actions and when to ask a human.

Robusta or our relay forwards the alert. **The model selects an SOP from the Holmes catalogue and requests** `fetch_skill`, using its name and description.

Permission to act is explicit

The SOP guides the investigation
Holmes loads the incident procedure: required evidence, repair conditions and cases that need a human. The SOP grants no system permissions.

Tools limit the available actions
Three fixed actions are available: start the HTTP service, restore file permissions or stop the identified CPU stress job. The repair tool checks each request. Memory pressure needs human intervention.



The repair tool requires an active incident, the expected evidence and the exact target. It limits attempts and verifies recovery. A refusal or failure requires a human handover.

An SOP guides checks and refusals to act

The SOP explains which checks to run, when to change direction and when to refuse an action.

Observation	Decision described in the SOP
API responds with `PermissionError`	Check the file, mount and permissions before DNS or networking. Evidence supports this priority.
Missing file or mount, or conflicting evidence	Leave the permissions procedure and review the possible cause. Another SOP may fit better.
Unavailable tool, unknown conditions or refused action	Hand over with the evidence and missing checks. Do not force an action or expand permissions.

After an action, check recovery. A failed check needs a report and human intervention.

Inside an SOP: act, verify or hand over

The procedure should define success and when the investigation stops. "Repair if needed" gives neither clear permission nor a recovery check.

What to define	Expected decision
Conditions for action	Define the required evidence, exact target and allowed tool before any change.
Limits and fallback	Set the maximum attempts and the response to a refusal or failure. Do not expand permissions during the incident.
Recovery	Repeat the health check after the action: HTTP status and expected content, beyond the command exit status.
Stopping and reporting	If unsure or making no progress, send facts, actions, unknowns and the next check to a named owner.

06

Why an investigation fails

The model depends on the observations it receives and the available resources. An investigation can stop because tools fail, context grows too large or memory runs out.

Our observed failures help explain the limits for steps, tokens and memory.

When an investigation runs without a conclusion

An incomplete investigation can have several causes. Before changing the model or increasing a limit, find the first failure in the tool calls and their results.

Symptom	What to check	Useful evidence
Turn limit reached	The investigation covers too much, or repeats checks without progress.	Timeline of calls and remaining possible causes.
Context too large	Long logs, crowded history or repeated compaction.	Input size, filters and summaries sent to the model.

Repeated checks may come from an unclear question, an unusable result or information lost from the history. The call timeline helps identify what needs fixing.

The turn limit bounds investigation cycles. Context size bounds the information available at each cycle.

Why limit investigation turns?

A configurable limit

`MAX_STEPS` limits investigation turns. One turn can request several tools. Each check should add new evidence or verify a change.

Our first trials used a limit of 8. This is not a universal Holmes rule.

The demo allows **12 turns**. This leaves room for several checks. It is neither a guarantee of success nor a target to reach.

What we observed

A broad investigation used all **8 turns** without a useful diagnosis.

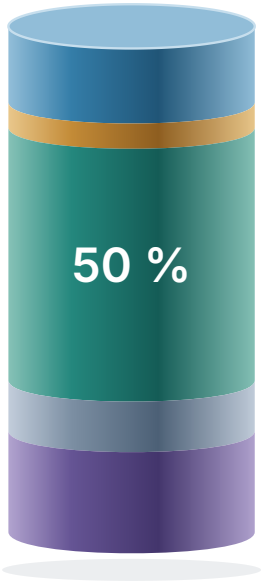
Increasing the limit to **16 turns** was not enough. Some tools failed and returned too much text.

Focused questions and bounded reads later produced useful reports. We needed usable observations as well as enough turns.

A larger budget cannot fix a failing tool or an investigation with an unclear scope.

Planning the context budget

At each call, Holmes sends the instructions, tool descriptions and useful history to the model. The model works with this content. It has no automatic long-term memory of the incident.



Instructions, SOPs and tool descriptions	**3,072** tokens 15% of context
Question and incident facts	**1,024** tokens 5% of context
Useful history and tool results	**10,240** tokens 50% of context
Headroom for input variation	**2,048** tokens 10% of context
Output reserve	**4,096** tokens 20% of context

Total budget: `max_context_size: 20480`. `max_tokens: 4096` applies per call, not to the whole investigation. Input shares are illustrative.

Compaction: summarising to continue



Making room
Holmes asks the model to summarise the history, then continues with that summary. It can investigate without sending all previous results again. This call uses time and tokens, **without taking an extra turn from** `MAX_STEPS`.

Possible loss of detail
The summary may leave out an exact error, a timestamp or a check that ruled out a possible cause. The model may repeat a check or return to an idea that earlier evidence had rejected.

Filtering results to useful evidence reduces the need for compaction.

Using fewer tokens while keeping the evidence

Before the investigation

Provide SOP names and descriptions, then load the relevant body with `fetch_skill`. Loading every full file would use context without helping the investigation.

Specify the resource, time window and expected behaviour. The model can then request focused checks instead of searching the whole system for any possible problem.

In tool results

Filter logs by service and time window. Limit rows and fields at the source. Reading a huge result and cutting it later still costs resources.

Remove repeated entries while keeping the error, date, target and trace ID. These details connect observations and help find the original data.

Report missing or truncated data so the model does not draw a false conclusion from it.

When tool results do not reach the model

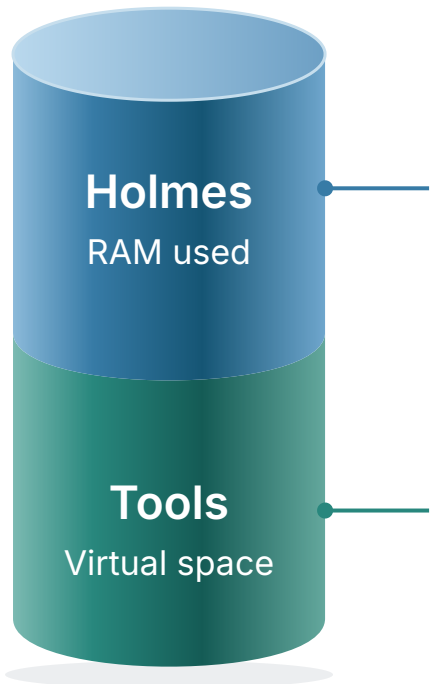
A tool can fail before returning any observation. Check how it runs and whether the components supporting the investigation are available.

Symptom	What to check	Useful evidence
Tool failure	A command cannot start, access is denied or the source is unavailable.	Exact error, exit code and the same check run alone.
Investigation interrupted	Container memory, model server or timeout.	Logs, OOM events and resource use by component.

A subprocess memory error, a Holmes container OOM and an unavailable LLM server need different fixes. A timeout alone cannot tell us which component failed.

Running the check alone helps test the tool before judging the model's answer.

Adjusting memory at the right layer



The Holmes container

The limit covers **RAM used** by Holmes and its tools (`mem_limit: 1536m` in the demo). Going over it can cause an **OOM**.

Options: filter results at the source, limit concurrent investigations, then size memory from measured peaks with some headroom.

Each tool process

`TOOL_MEMORY_LIMIT_MB` limits the **virtual address space** per command. The Go runtime in `kubectl` may reserve a large space at startup without using that much RAM.

Option: run the check alone. Adjust this limit if a failed memory reservation is confirmed.

LLM server RAM/VRAM: measure model and cache use, then adjust context, concurrency, quantisation or capacity.

Demo: RAM, CPU and a missing file

01 RAM: diagnose and report

`task fault:memory` causes memory pressure. Holmes sends a report to ntfy and prepares a human handover. No tool allows it to stop the memory consumer.

02 CPU: identify, stop and verify

`task fault:cpu` starts a stress job. Holmes identifies it and calls the allowed stop tool. The tool checks CPU recovery and alert resolution, then Holmes reports the result.

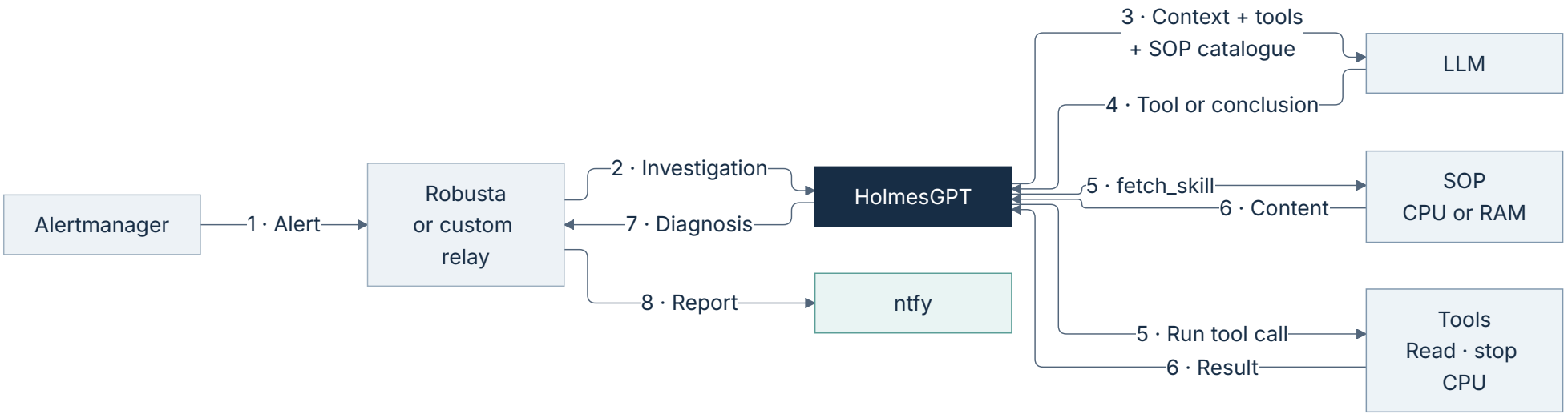
03 Missing file: FastAPI runs, one endpoint fails

`task fault:file` leaves FastAPI and its home page running, but `/api/check` returns HTTP 503. Holmes connects the logs and traces to the missing file and requests human intervention.

Grafana Alerts Holmes reports Jaeger

The same investigation, with one extra action tool

RAM and CPU share the investigation loop and read tools. Each alert has a relevant SOP.



RAM: diagnosis and human intervention. **CPU:** `stop_demo_cpu_stress` stops the identified job, then checks CPU below 85% and alert resolution.

The SOP guides diagnosis. An allowed tool makes action and verification possible.

Back to the recovery check

What we receive in ntfy

Real notification from 4 November, shortened English extract

Holmes demo topic `holmes` · 11:36

Holmes — DemoCpuBusy

Facts. Host CPU is at 99.98%. The `labstack-demo-cpu.service` job is active and uses 98.4% of CPU. FastAPI returns HTTP 200.

Identified cause. The `stress-ng` job saturates the VM's CPU.

Action. Call to `stop_demo_cpu_stress` to stop this job.

> **Tool: recovery verified.** CPU job stopped, host CPU below 85% and `DemoCpuBusy` alert resolved.

Incident: `340f34c77e58`